FORUM NOKIA

Presence Application Development Guide

Version 1.0; April 30, 2003

Presence

NOKIA

Contents

Change History

April 2003	V1.0	Initial document release

Disclaimer

The information in this document is provided "as is," with no warranties whatsoever, including any warranty of merchantability, fitness for any particular purpose, or any warranty otherwise arising out of any proposal, specification, or sample. Furthermore, information provided in this document is preliminary, and may be changed substantially prior to final release. This document is provided for informational purposes only.

Nokia Corporation disclaims all liability, including liability for infringement of any proprietary rights, relating to implementation of information presented in this document. Nokia Corporation does not warrant or represent that such use will not infringe such rights.

Nokia Corporation retains the right to make changes to this specification at any time, without notice.

The phone UI images shown in this document are for illustrative purposes and do not represent any real device.

License

A license is hereby granted to download and print a copy of this specification for personal use only. No other license to any other intellectual property rights is granted herein.

Presence Application Development Guide

Version 1.0; April 30, 2003

1 Introduction

Presence is a hot topic in mobile communications, and most terminal and network infrastructure vendors are busy introducing their presence solutions. For application developers, presence provides attractive possibilities for creating new services and strengthening existing ones. But what is it all about?

The following document provides an overview of presence, how presence information can be used, and what kinds of applications can be developed based on presence. The information is especially appropriate for developers interested in building presence-based applications; accordingly, it offers an overview of all key issues that must be understood in order to begin to develop presence applications. This document can also be used as a self-teaching guide.

Topics range from standardization, the presence concept, and Web services, to the properties of the Nokia Presence Server Web Service Interface. A brief overview of presence -enabled terminals is also included, as well as a look at tools that will assist with presence application development.

2 Standardization Status

Instant Messaging and Presence are moving from the desktop to the mobile domain. To support this evolution, the Wireless Village initiative was formed in April 2001 by Ericsson, Motorola, and Nokia, to build a community around new and innovative mobile Instant Messaging and Presence Services (IMPS). The Wireless Village was then consolidated with the Open Mobile Alliance (OMA) on October 1, 2002. Future work on the former Wireless Village initiative specifications is now an activity within the OMA Instant Messaging and Presence Service Working Group (OMA IMPS WG).

Through its supporters, OMA IMPS WG aims to build a vibrant community of users and global business partners where the Internet and wireless domains converge. OMA IMPS WG will define a set of universal specifications for mobile IMPS, as well as procedures and tools for testing conformance and interoperability. The specifications will be used for exchanging messages and presence information between mobile devices, mobile services, and Internet-based instant messaging services. In short, OMA IMPS is about building the market, making the specifications, and ensuring interoperability. For further information on OMA IMPS WG and specifications, see http://www.openmobilealliance.org.

3 Introduction to the Presence Concept

Presence is the key enabling technology for OMA IMPS. In the desktop world, users have been able to announce their status to authorized recipients. Using this status information, users have been able to start communicating with those people that are connected to the network and available for communication.

Presence promises to be even more powerful in the mobile world due to the nature of mobility itself and the close association between consumers and their terminals. Presence enhances all mobile communication, and presence information will be available in the terminal everywhere that communication is generated — phone book, quick dial, redial, call logs. Nokia expects presence-specific as well as presence-enriched applications and services in the near future. A typical example of a presence-specific application would be a corporate phone book with embedded presence information making it dynamic.

In the OMA IMPS model, presence takes on a richer meaning. It includes client device availability (my phone is on/off, in a call), user status (available, unavailable, in a meeting), location, client device capabilities (voice, text, GPRS, multimedia), and searchable personal status such as mood (happy, angry) and hobbies (football, fishing, computing, dancing). Since presence information is personal, it is only available according to the user's wishes – access control features put control of user presence information in the user's hands.

The following sections describe the presence concept and provide use cases for application developers.

3.1 Presence in Practice

Presence is a dynamic profile containing information about the user's availability, mood, intentions, contact preferences, and so on. The presence service allows users and applications to obtain information such as how to contact the user. Some of the user contact information can be relatively static, for example:

- Telephone number (home and/or business)
- E-mail address
- Street address

Other user contact information may be more dynamic or transitory:

- Current "sessions" (phone call, browsing, retrieving streaming content, etc.)
- User context (in a meeting, on holiday, etc.)
- User willingness to participate in certain types of communication (short messaging, multimedia messaging, phone calls, etc.)

The relatively static information can be used for services such as directory services that can be accessed from home or corporate PCs. The more dynamic presence services can be used for buddy-list types of services, where a friend can look at his/her address book and see the availability and context of a friend to be contacted. Dynamic presence information also enables the user to create a virtual identity, for example, with logos, mood impressions, and status texts. The Presence Web Service Interface enables service and content providers to purchase the information they need to make their services more personal and to increase the value of mobility.

3.1.1 Presence attributes

So, what kinds of details does the presence information contain? OMA IMPS has defined a set of presence attributes that form the basis for presence information. These attributes are listed in the following table.

Table 1: Presence attributes

Attribute Name	Description
OnlineStatus(S)	Indicates if the client device is logged on a Presence Server
Registration	Indicates if the client device is registered in the mobile network
ClientInfo	Information about client, e.g. client model and manufacturer, version of the client

TimeZone	Local time zone of the client device
GeoLocation	Geographical location of the client device
Address	Address of the client device
FreeTextLocation	Free text description of the location of the user
PLMN	PLMN code of the network the client device is registered to
CommCap	Communication capabilities of the client
UserAvailability(S)	Availability of the user for communication
PreferredContacts	Contact preferences of the user
PreferredLanguage	Language preference of the user
StatusText(S)	User-specified status text
StatusMood	Mood of the user
Alias	Alias name for the user
StatusContent	Media information for user status
ContactInfo	A vCard for the user

The presence service attributes can be relegated to two main categories depending on the nature of information they hold: client status attributes and user status attributes.

3.1.1.1 Client Status Attributes

Client status attributes describe the status of the running presence client software as well as the hardware device. They include presence attributes that describe the status of the presence client and device in relation to the mobile or fixed network as well as more detailed information about the client itself, such as version and capabilities. The network status of the client includes the registration and online status of the client to the network as well as the location and address information of the client.

3.1.1.2 User Status Attributes

User status attributes describe the status of the presence user. They include attributes that describe user availability and preferred contact methods as well as the user's contact information. User status attributes also include information that may be used to describe textual free-format status, status with content, and the emotional state of the user, such as mood.

3.1.2 Contact lists

A contact list is a user-maintained list of presence users in the presence service element. In Wireless Village specifications, the contact list is used for various purposes: as a distribution list when sending instant messages or subscribing presence, as a mechanism for proactive presence authorization, etc.

Users can manage multiple contact lists for various purposes, such as a list of friends and a list of colleagues. The management of contact lists includes features for creating, deleting, and editing contact lists as well as the ability to obtain a list of contact lists. Users can also modify and retrieve the contents of a contact list from a presence service element. The visibility of presence attributes on a certain contact list can be managed by the user. For example, users in the "family" list might have more presence attributes visible than users in the "friends" list. Figure 1 summarizes the concept.

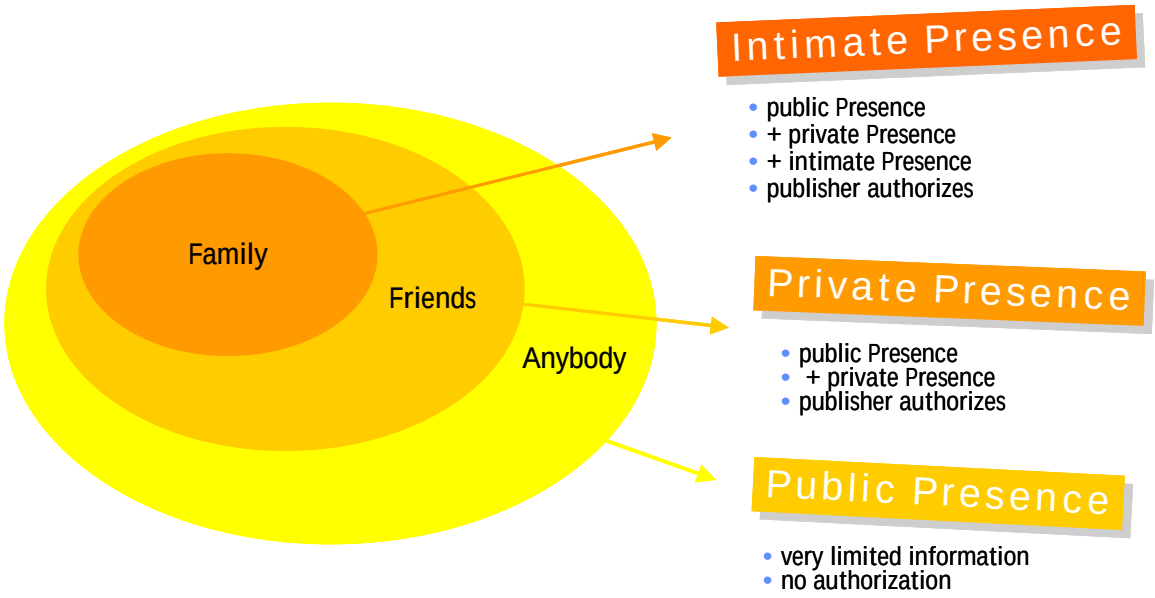

Figure 1: Different contact lists

3.2 Presence in Terminals

Presence has substantial visibility in terminals. For example, it can be used to enhance a terminal's phone book by providing dynamically changing status information about user contacts, it can be used in game applications to locate a suitable co-player, etc.

If dynamic presence information is brought to a terminal's phone book, it can be used to determine the most appropriate communication method. Let's say, for example, that Jim is in a meeting and is unable to answer a voice call. If his presence information (in this example availability=meeting) is in John's phone book, John can decide to send an SMS instead of a voice call. With presence information in a terminal's phone book, the user can decide, before communicating, what is the most suitable way to communicate (e.g., voice call, messaging) and whether or not it's even feasible to contact the person (e.g., because he is unavailable). Some people simply enjoy knowing what their friends and family are doing without having to contact them.

3.3 Presence Network Architecture

The Presence Server, which enables the use of presence information in the network, is located in the operator's intranet, as illustrated in Figure 2. It is connected to the packet network via the WAP gateway or HTTP proxy (it depends on the terminal implementation, whether the OMA IMPS protocols are transferred over WAP or HTTP).

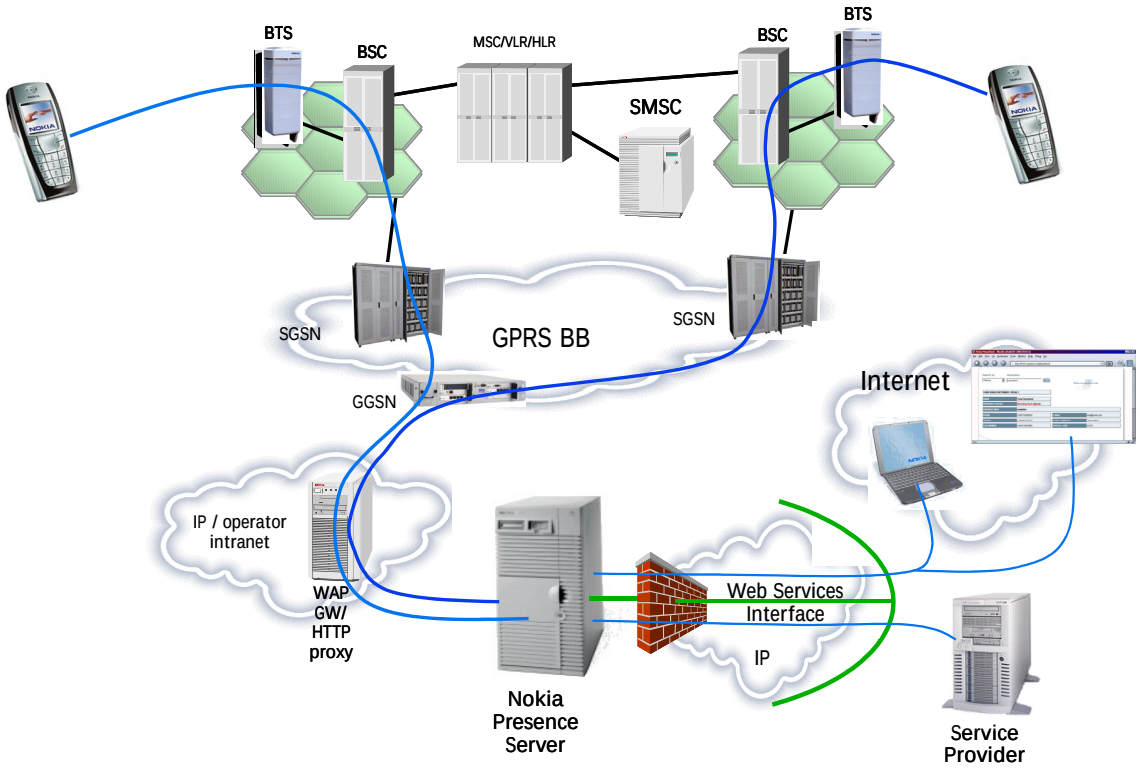

Figure 2: The Presence Server in GPRS network

Let's take a closer look at what happens in the network during the various presence transactions. For a presence service subscriber there are two ways to get presence information about another user: either by subscribing to the other user's presence information or by performing a one-time fetch. If s/he subscribes to another user's profile as a watcher, s/he will be notified each time the presence changes, resulting in up-to-date information about the other user's presence. In a one-time fetch, the user makes a query to the Presence Server for the other user's presence and in the response receives the presence values. If the user wants presence information again, s/he must either perform the fetch again or subscribe to the profile. Figure 3 illustrates the transactions that take place in both scenarios.

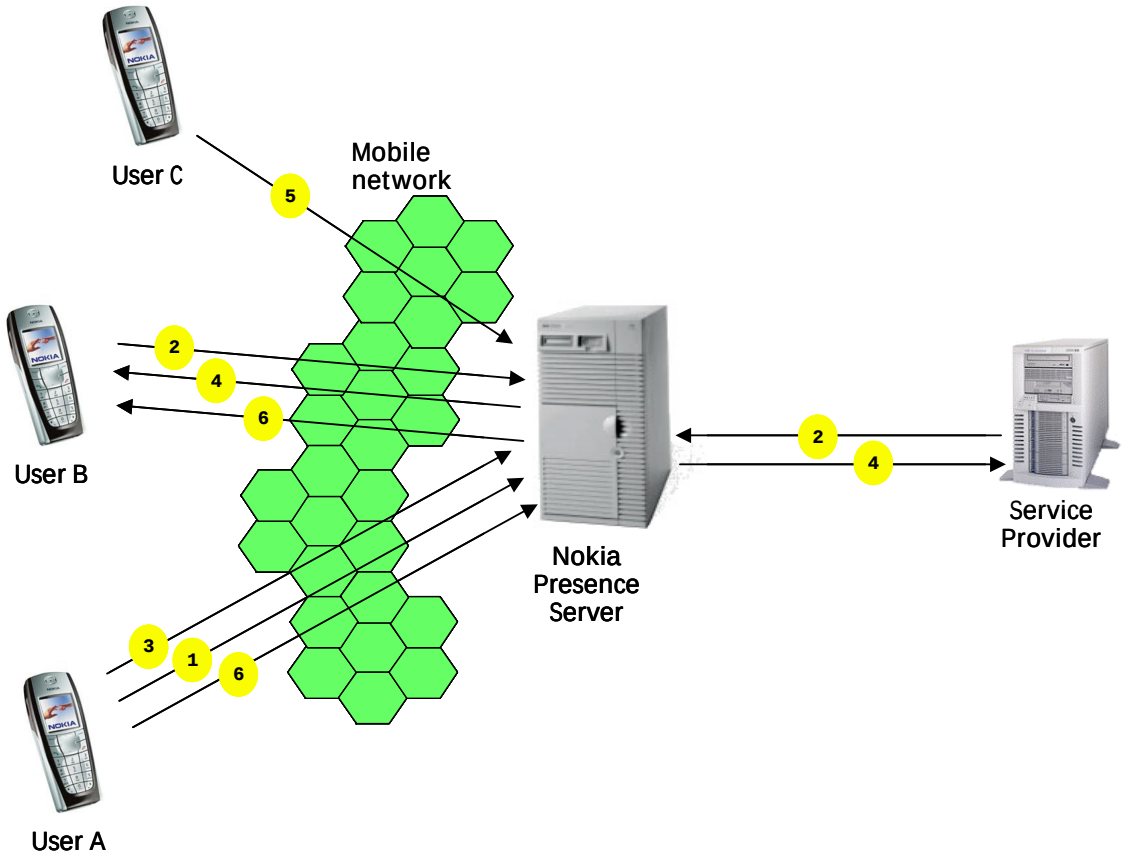

Figure 3: Presence usage example

1. Mobile terminal user A publishes presence information on the Nokia Presence Server.

2. User B and the Service Provider subscribe to user A's presence information.

3. User A can update presence information to the Nokia Presence Server via the terminal, or network applications can make updates.

4. The users who have subscribed to user A's presence information receive the updated information.

5. Other users (user C) can also fetch A's presence information when needed on a one-time basis without subscribing to updates.

6. Contact lists are used for authorization. User A may authorize user B to subscribe to his/her private presence information.

3.4 Presence Application Examples

Presence enables many kinds of applications to be developed, on both the server and terminal sides. These application opportunities can be divided into three groups:

- Presence-enabled applications, which require presence technology to function properly
- Presence-enhanced applications that are using presence to create added value for end users
- Presence-based information channels in which the applications use presence technology to deliver information to the end users

Server-side applications can be divided into groups depending on the type of application.

Service types: Person-to-person applications

- Content-to-person applications
- Corporate applications

Person-to-person applications enhance communication between two or more users by providing useful availability information. They also allow end users to express themselves and create a virtual identity, for example, with individual status text and logos. Example applications include:

- Presence-enhanced contacts (in Nokia terminals)
- Mobile messaging, e.g., combined presence and MMS service where MMS messages are sent to subscribers based on their mood
- Self expression, virtual identity, logo service
- Mobile gaming, e.g., in chess, presence can be used to indicate who has the next move

Content-to-person applications enable new kinds of information and content-sharing possibilities. With presence technology, content and service providers can deliver information to the terminal in real time, for example, delivering the score of a football game, weather forecasts, advertisements, etc. This information is available to the end user immediately—no action is needed to retrieve it. The following kinds of service ideas are examples of content-to-person applications:

- Football game information channel
- Multi-story car-parking company publishing real-time information about its available downtown parking
- Operator advertising services
- Fast-food restaurant advertising the product of the day
- Radio station updating the name of the current song and offering a link where the user can get the song (e.g., as a ringing tone)
- Stock exchange rates

Corporate applications enhance employee effectiveness. With corporate applications, presence information can be integrated with the corporate phone book to provide information about availability or it can be integrated with the user's corporate calendar so that others will know where the employee is. Other corporate application examples are:

- Corporate task list
- Resource management

4 Web Services

Web Services is a hot topic in current Internet technology. It, too, is expanding into the mobile service domain. Different companies and standardization bodies seem to have slightly different definitions for the term "Web services," although all are fundamentally similar. This chapter will briefly describe the Web services, what they are, and how they work.

4.1 Introduction

Web services are well-defined, published protocol interfaces through which businesses offer electronic services. The IT industry has succeeded in automating most of the operational processes taking place inside corporations. Web services promise the tools to automate business-to-business relationships over the Internet and are widely viewed as the next step in the Internet's evolution. The past year has seen the emergence of Web services protocols for service-oriented electronic business. Web services provide a common and interoperable way to define, publish, and use mobile value-added services, or to provide mobile access to existing Internet content. They build on existing and emerging technologies such as Extensible Markup Language (XML), Simple Access Object Protocol (SOAP), Web Services Description Language (WSDL), and Hypertext Transfer Protocol (HTTP). In the future, Universal Description, Discovery, and Integration (UDDI) may also play a significant role in creating and implementing a directory of Web services.

Many of the core security features and implementations are based on existing and proven technologies such as Public Key Infrastructure (PKI), Secure Sockets Layer (SSL), or Transport Layer Security (TLS). Web services are also independent of the operating system on which they run and enable different systems to communicate effortlessly with each other – as long as the Web services have been implemented according to the standards (for more information about Web services, see http://www.openmobilealliance.org).

The Web services architecture consists of three entities:

- A service broker that acts as a look-up service between a service provider and a service requestor
- A service provider that publishes its services to the service broker
- A service requester that asks the service broker where to find a suitable service provider and that binds itself to the provider

Figure 4 offers a simplified illustration of the functionality of Web services. The Service Provider is publishing its service to the Service Broker. The information contains instructions on how to contact the service and how to format the message in order to use the service. The Service Requester that needs a service is asking the Service Broker to provide contact information for a Service Provider that corresponds to the Service Requester's needs. As a result of the inquiry, the Service Broker will send the Service Provider's contact information to the Service Requester and, based on that information, the Service Requester will contact the Service Provider using the standardised mechanism.

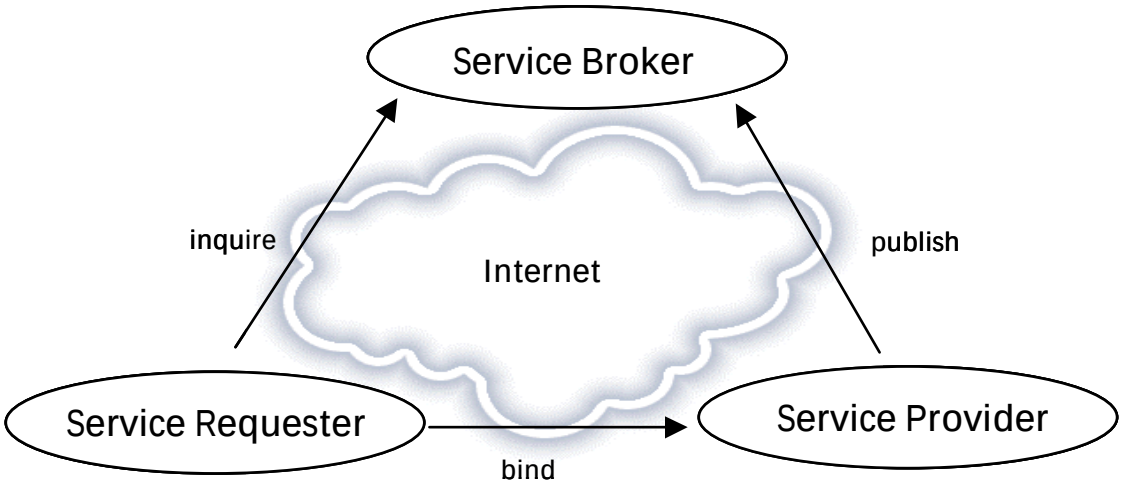

Figure 4: Web services simplified architecture

Here's a practical example. John wants to redecorate his apartment but he can't do it by himself, so he decides to look in the Yellow Pages to find a suitable decorator. Bob has a decorating company. He has published his contact information in the Yellow Pages. John is exploring the Yellow Pages and Bob's advertisement pops up. John decides that Bob's company is the most suitable and he calls Bob to redecorate his apartment.

If we mirror the given example into Web services, the roles can be easily detected. John is the equivalent of the Service Requester needing help with his decoration. He goes to the Yellow Pages to find a suitable decorator. Bob, who offers decorating services and publishes his contact information in the Yellow Pages, equates to the Service Provider. Here, the Yellow Pages act as the Service Broker, providing search services for the Service Requester who needs to find the most suitable Service Provider for his needs.

4.2 Web Services – Beneath the Surface

Next we will take a closer look at the technology used in Web services. Figure 5 illustrates the protocols used in Web services and how they relate to each other.

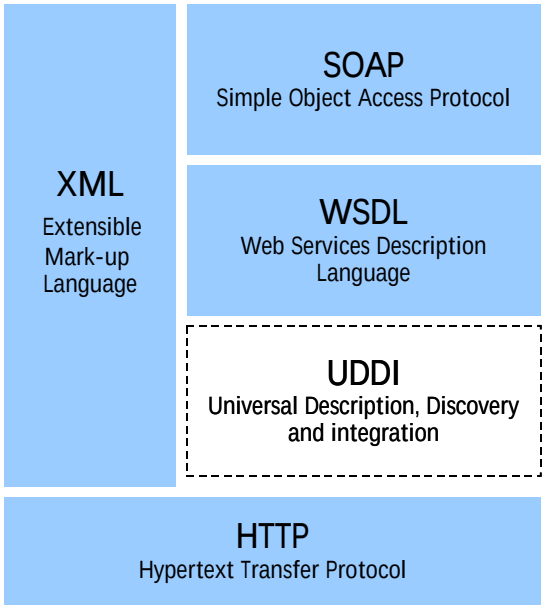

Figure 5: Web service protocols

- Hypertext Transfer Protocol (HTTP) acts as a transport for the Web services. It takes care of transferring the Web services related information flows from client-to-server and vice versa.
- Extensible Markup Language (XML) is the language for defining data and how to process it.
- Simple Object Access Protocol (SOAP) specifies how to create a message envelope, header, and body to be transported between service consumer and producer. In short, it describes how to format the message.
- Web Services Description Language (WSDL) specifies how a Web service can be accessed, what inputs are required, and what outputs are generated. In other words, it describes how to describe a Web service so that it can be used. Universal Discovery, Description, and Integration (UDDI) describes how Web services can be listed by name, product, location, or services they offer in online business registries. It offers the white, yellow, and green pages.

In a nutshell, HTTP is the bearer for the information in the Web services technology. SOAP is used to create and formulate a message so that it can be transported over HTTP and interpreted by both the

service provider and the service requestor. WSDL is used in the service broker to define a certain service provider's service; basically it contains information on how to contact that particular service and how to use it to get the needed outputs. UDDI is the “service broker’s information repository,” which describes all of the service provider's service information and offers a “search service” for service requesters to find the most appropriate service provider.

5 Presence Web Services Interface

The Presence Web Services Interface (WSI) provides access to external applications to retrieve and update the presence data of Nokia Presence Server users. The interface exposes presence service elements over Web services.

The Presence WSI can be used for service-initiated presence requests to the Nokia Presence Server as well as for Nokia Presence Server-initiated notifications to the service application. By offering the Presence WSI to service providers, the operator enables a rapid and standards-based deployment of a wide variety of services offered by different service providers. In the future, a WSI similar to the one in the presence solution will be deployed on a number of other platforms.

The Presence WSI provides possibilities for integrating other systems and services with the Nokia Presence Server. This API makes it possible for other applications to use presence information in, for example, a corporate phone book or scheduling and time management solutions. In addition, the Presence WSI can be used for network-based or triggered presence updates and monitoring for users or other entities that may have associated presence data. For more detailed information about Presence WSI, please see the *Nokia Presence Server Service Developer’s Guide* [1].

5.1 Accessing the Presence Server

Before the presence application can be deployed into an operator's live mobile network, the following connection information must be supplied by the operator or presence service provider:

- The user ID to be used by the application. In order to publish presence information or request another user's presence information, an application needs a user ID to send requests through Presence WSI.
- An HTTP authentication user name for the application.
- An HTTP authentication password for the application.
- A URL to be used by the application for accessing Presence WSI.

5.2 Authentication and Authorization

Authentication and authorization play an important security role in Presence WSI. When developing WSI applications, the following security aspects must be taken into account:

- *Identification and authentication*: The Nokia Presence Server can request a user name and password from an application by using HTTP Basic or HTTP Digest Authentication. The preferred authentication method can be set for each application. When the Nokia Presence Server is placing a request to an application, it can authenticate itself against HTTP Basic and HTTP Digest Authentication prompts.
- *Authorization*: The Nokia Presence Server authorizes an application by comparing the credentials against Nokia Presence Server configuration information.
- *Delegation*: When using Presence WSI, an application is acting on the behalf of an OMA IMPS user.

5.3 Presence WSI Transactions

Presence WSI is a public interface through which applications are able to use presence services. These services provide functionality for presence subscription management, presence fetch and update, contact list management, attribute list management, and search. Here we will look at what is contained in those categories and how an application developer can take advantage of them. For a detailed description of Presence WSI transactions, see the *Service Developer's Guide* [1]

5.3.1 Presence subscription management

Presence applications and service providers can manage their subscriptions to other users. They can subscribe to another user's presence in order to be notified each time the user's presence changes. Users can also query the list of users to whose presence information they have subscribed as well as request the list of those users that are his own watchers. The watcher list relays all the watchers that have subscribed to the user's presence.

5.3.2 Presence fetch and update

With presence fetch and update, subscribers can request a user's presence information on a one-time basis, update their own information to the Presence Server, and receive notifications from the users' presences to which they have subscribed.

5.3.3 Contact list management

Presence subscribers can create their own contact lists. The lists can be used to define the visibility of certain presence attributes differently to different user groups. For example, a user can have different contact lists for work colleagues and family. S/he can make sure that during the weekend, colleagues see only that s/he is unavailable, but his/her family sees that s/he is playing tennis. In addition to contact list creation, users can also delete and manage lists. Users are also allowed to request all the members of their contact lists.

5.3.4 Attribute list management

Users and applications can have several contact lists for different purposes. In addition, each of the contact lists can have different attribute lists. So, in practice, the application or user can add different attributes that are visible to the members of that particular contact list. Attribute lists are not just limited to contact lists — an attribute list can also be created for a single user or application. The use of attribute lists is limited to one attribute list per contact list or per user (so they cannot have more than one attribute list). The relation between contact lists and attribute lists is clarified in Figure 6.

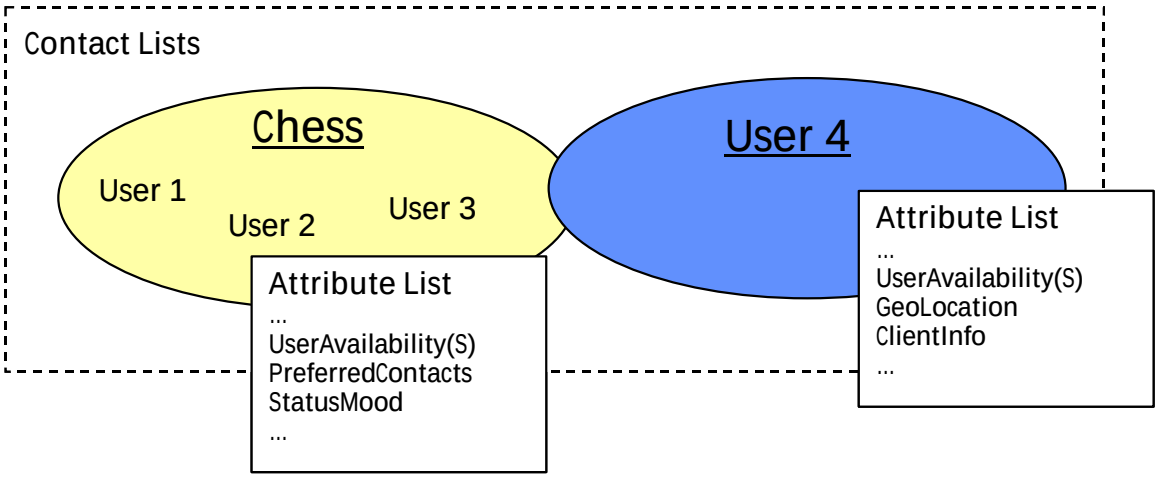

Figure 6: Relation between contact lists and attribute lists

Through the Presence WSI it is possible to create and delete attribute lists for contact lists or for single users. An application is also allowed to perform a get-attribute-list request in order to retrieve attributes it associates with a specific contact list(s) and/or user(s), and/or the default attribute list.

5.3.5 Search

Applications are allowed to perform a search operation through the Presence WSI. A search from the Presence Server can be based on various user properties containing the user ID, MSISDN, online status, user's first name, user's last name, user's e-mail address, or user's alias. If the search operation is successful, the Presence Server will always return the user ID(s) of the matched users. In the Presence WSI it is also possible for an application to stop an already activated search if, for example, the application no longer needs the results of the operation.

5.4 Presence WSI SOAP Examples

Below is an example of a Presence WSI SOAP request/response for Get Presence Request:

```
==== Request ====
POST /axis/services/Passport HTTP/1.0
Content-Type: text/mol; char set=utf-8
Accept: application/soap+xml, application/dime, multipart/related,
text/*
User-Agent: Axis/1.1beta
Host: localhost
Cache-Control: no-cache
Pragma: no-cache
SOAPAction:
"http://www.nokia.com/schemas/Presence/v1.0/?operation=GetPresence"
Content-Length: 588
Authorization: Basic cHNodHRwdXNlcjpwc2h0dHBwd2Q=

<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
 <soapenv:Body>
  <GetPresenceRequest
xmlns="http://www.nokia.com/schemas/Presence/v1.0/">
   <UserId>wsiappuser</UserId>
   <UserIdTable>
    <UserId>requesteduser</UserId>
   </UserIdTable>
   <AttributeNameTable>
    <AttributeName>
     <Name>OnlineStatus</Name>
    </AttributeName>
   </AttributeNameTable>
  </GetPresenceRequest>
 </soapenv:Body>
</soapenv:Envelope>


==== Response ====
HTTP/1.1 200 OK
Pragma: No-cache
Cache-Control: no-cache
```

```
Expires: Thu, 01 Jan 1970 00:00:00 GMT
Content-Type: text/xml; charset=utf-8
Date: Tue, 25 Mar 2003 07:48:36 GMT
Server: Apache Coyote/1.0
Connection: close

<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
 <soapenv:Body>
  <GetPresenceResponse
xmlns="http://www.nokia.com/schemas/Presence/v1.0/">
   <Status>
    <StatusCode>1000</StatusCode>
    <StatusText>Operation was successful</StatusText>
   </Status>
   <AttributeAssociationTable>
    <AttributeAssociation>
     <UserId>requesteduser</UserId>
     <PresenceAttributeTable>
      <PresenceAttribute>
       <AttributeName>
        <Name>OnlineStatus</Name>
        <Namespace>http://www.wireless-village.org/PA1.1</Namespace>
       </AttributeName>
       <UpdateTime>2003-03-25T07:48:28.671Z</UpdateTime>
       <Qualifier>true</Qualifier>
       <ValueTable>
        <NamedValue>
         <Name>PresenceValue</Name>
         <Value>F</Value>
        </NamedValue>
       </ValueTable>
      </PresenceAttribute>
     </PresenceAttributeTable>
    </AttributeAssociation>
   </AttributeAssociationTable>
  </GetPresenceResponse>
 </soapenv:Body>
</soapenv:Envelope>
```

6 Nokia 6220 and Presence-Enhanced Contacts

The phone book in a mobile phone is currently used to manage contacts. Users can store static contact information in memory, which allows them to communicate via voice calls, SMS, MMS, e-mail, etc. The phone book of today's mobile phone resembles the illustration in Figure 7:

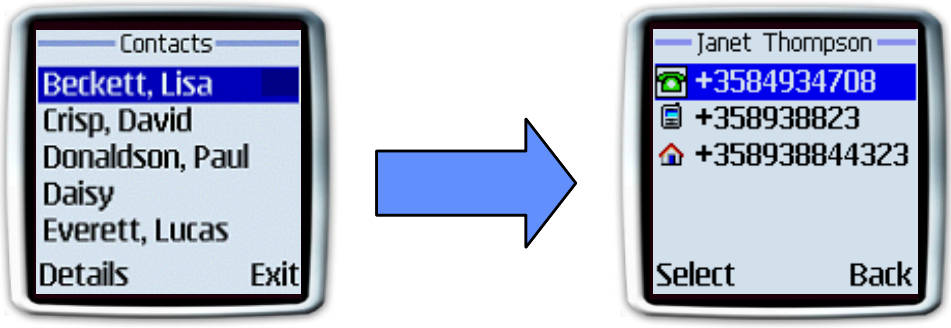

Figure 7: Terminal contacts without presence functionality

Users have no way of knowing whether a friend or colleague can answer a call, what their availability is, and whether it would be better to send a message. With presence service, all of this is possible, thanks to up-to-date presence information in their phone book. Figure 8 illustrates the concept of presence-enhanced contacts.

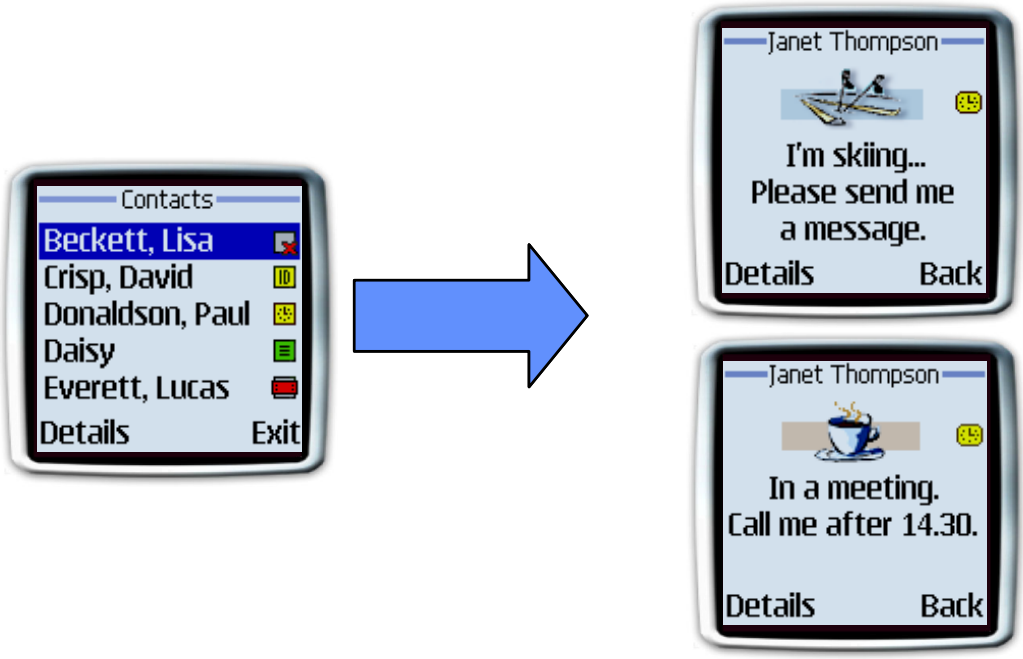

Figure 8: Presence-enhanced contacts

With presence-enhanced contacts, users can actually decide, before communicating, what is the most suitable way to communicate and whether or not it's worth trying to contact the person. Some people just enjoy knowing what their friends and family are doing without having to contact them.

Presence allows users to control the presence information they are publishing. It also allows them to change related settings and manage the lists of people who are viewing their published presence. Presence-enhanced contacts allow users to view the presence of a *subscribed memory entry* and to add a WV ID to a phone book entry. The presence-enhanced contacts capability also attempts to match the ID of a viewer to a memory entry in the phone book.

With presence-enhanced contacts in the Nokia 6220, the user is able to:

- Connect to and disconnect from the Presence Server.
- View and edit the *presence service* connection settings.

- View the presence status and the presence of a *subscribed memory entry*.
- View who is subscribed to his/her published presence.
- Select his/her own presence availability status from the following:
  - Available
  - Not available
  - Discreet
- Personalize the presence status message. The status message is a set of text that the user can personalize to accompany his/her status logo. Text can be up to 50 characters in length.
- Select a presence status logo. Gallery is used for storing and retrieving logos to be used by the presence application.
- Select a publishing level. The user can control whether to publish his/her presence status and which group of people to publish to. The options are:
  - No-one
  - Private only
  - Private and public
- Assign private or blocked status to phone book entries, active, and subscribed viewers.
- Unsubscribe to a memory entry.
- Select whether to link his/her presence status to profiles.
- Request *presence status attribute* on a one-time basis.

When using presence, the phone supports General Packet Radio Service (GPRS) as the bearer and Hypertext Transfer Protocol (HTTP) as the transport protocol. A user will have to have a *presence service* account and his/her Wireless Village ID from a service provider to fully use this feature in the phone.

7 Tools for Presence Application Development

Several tools are available to help in developing presence applications. Currently, tools available to assist in server-side application development are part of the Nokia Mobile Server Services SDK 1.1 (NMSS SDK). This SDK is available from the Forum Nokia Web pages, http://www.forum.nokia.com. The following sections take a look at the presence tools contained in that SDK.

7.1 Presence Server API and Java™ Libraries

As part of the Presence SDK, Nokia is shipping out a presence Java™ technology library. This library can be used to construct a Presence WSI request and also to handle the response the Presence Server is sending. The API enables fast, easy application development by providing WSI functionality in the form of Java technology libraries. With this library, all of the operations the WSI is implementing can be utilized.

7.1.1 Example: Getting presence attributes

The following example shows how to use the Presence API to get the presence attributes:

1. Prepare the list of users (or leave it null if only contact list attributes are requested):

```
String [] usrs = {"user1","user2","user3"};
```

2. Prepare list of contact list IDs (or leave it null if only user attributes are requested):

```
String [] lists = {"list1","list2"};
```

3. Prepare the presence attributes you want to get from the server:

```
PresenceAttributeIdentifier[] pas = new PresenceAttributeIdentifier [6];
```

Do this in one of the following ways:

4. Instantiate the specific attribute object:

```
pas[0]=new UserAvailability();
pas[1]=new StatusText();
pas[2]=new OnlineStatus();
```

5. Use the generic object:

```
pas[3]=new PresenceAttribute(PresenceConsts.ATTR_ADDRESS);
pas[4]=new PresenceAttribute(PresenceConsts.ATTR_GEO_LOCATION);
pas[5]=new PresenceAttribute(PresenceConsts.ATTR_FREE_TEXT_LOCATION);
```

Passing null instead of list of attributes indicates that all attributes for given users are to be returned. In this example, the following attributes should be retrieved: UserAvailability, StatusText, OnlineStatus, Address, GeoLocation, and FreeTextLocation.

```
Presence []attribs =
            engine.getPresence(requestorId,usrs,lists, pas);
```

More examples can be found in the *Java™ API Developer's Guide for Nokia Presence Server* document.

7.2 Presence Server Emulator

The server-side presence application developer must interface with the Presence Web Service Interface. In order to verify that the application is compliant with the Presence Server, it is recommended to test the application before commercial deployment. First steps with testing can be taken by using the Presence Server Emulator that mimics the entire functionality of the Presence WSI, enabling application developers to test their applications without connection to a real Presence Server. From the Presence Server WSI emulator, the following functionalities can be discovered:

Applications can send any of the subscription management, fetch and update, contact list management, attribute list management, or search requests to the emulator. In return the emulator will respond with the corresponding response and the correct values. In addition, presence update notifications can be sent from the emulator to the application.

The user of the emulator can select the response method to the request from three different modes:

- Manual mode, in which the user can select the parameters and values that the emulator is sending to the application in the response
- Automatic mode, in which the emulator responds with the correct response and with some default parameters

- Performance mode, in which the emulator will not update the graphical user interface and sends the response immediately without allowing the user to modify the response. This offers a powerful way to the test the efficiency of the application.

The emulator also provides a set of monitoring and management functionalities that are useful in application development. Via the emulator's graphical user interface it is easy to keep track of the tests performed and their results, as well as analyse the logs from communication between the emulator and the application.

8 Terms and Abbreviations

Term or Abbreviation	Description
API	Application Programming Interface
HTTP	Hypertext Transfer Protocol
MMS	Multimedia Messaging Service
OMA	Open Mobile Alliance
PEC	Presence-Enhanced Contacts
SDK	Software Development Kit
SOAP	Simple Object Access Protocol
UDDI	Universal Description, Discovery, and Integration
WSDL	Web Services Description Language
WSI	Web Services Interface
WV	Wireless Village
XML	Extensible Markup Language

9 References

[1] Nokia Presence Server, Service Developer's Guide

[2] Nokia Mobile Server Services SDK, Emulator User Guide for Presence

[3] Nokia Mobile Server Services SDK, Java™ API Developer's Guide for Nokia Presence Server

Build Test Sell

Developing and marketing mobile applications with Nokia

Go to Forum.Nokia.com

Forum.Nokia.com provides the tools and resources you need for content and application development as well as the channels for sales to operators, enterprises, and consumers.

Forum.Nokia.com

Download tools and emulators

Forum.Nokia.com/tools has links to tools from Nokia and other industry leaders including Borland, Adobe, AppForge, Macromedia, Metrowerks, and Sun.

Forum.Nokia.com/tools

Get documents and specifications

The documents area contains useful white papers, FAQs, tutorials, and APIs for Symbian OS and Series 60 Platform, J2ME, messaging (including MMS), and other technologies. Forum.Nokia.com/devices lists detailed technical specifications for Nokia devices.

Forum.Nokia.com/documents
Forum.Nokia.com/devices

Test your application and get support

Forum Nokia offers free and fee-based support that provides you with direct access to Nokia engineers and equipment and connects you with other developers around the world. The Nokia OK testing program enables your application to enjoy premium placement in Nokia's sales channels.

Forum.Nokia.com/support
Forum.Nokia.com/ok

Market through Nokia channels

Go to Forum.Nokia.com/business to learn about all of the marketing channels open to you, including Nokia Tradepoint, an online B2B marketplace.

Forum.Nokia.com/business

Reach buyers around the globe

Place your applications in Nokia Tradepoint and they're available to dozens of buying organizations around the world, ranging from leading global operators and enterprises to regional operators and XSPs. Your company and applications will also be considered for the regional Nokia Software Markets as well as other global and regional opportunities, including personal introductions to operators, on-device and in-box placement, and participation in invitation-only events around the world.

Forum.Nokia.com/business